

SHRIRAM ALAGAPERUMAL

THIRUMAVALAVAN

+91-9944234127 | Shriram_APT@outlook.com | LinkedIn | GitHub

EDUCATION

Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat

(2024–Present)

- Bachelor of Technology (B.Tech) in Computer Science and Engineering
- CGPA: 8.45/10 | Coursework: Data Structures, Computer Organization, Microprocessors and Interfacing, Networks
-

TECHNICAL SKILLS

Languages: C, C++, Rust, Python, x86 Assembly

Operating Systems & Platforms: Linux, x86 Architecture

Development Tools: Git, GitHub, GDB, QEMU, Make, Bash, GCC, NASM

Libraries & APIs: POSIX, pthreads, ncurses

PROJECTS

⇒ **WM File Manager | C, ncurses, POSIX Processes/ Filesystem, Linux (2025)**

- Engineered a feature-rich **terminal file manager** in C with an interactive **ncurses-based UI**, supporting **file navigation**, clipboard operations, renaming, deletion, and executable/script launching.
- Leveraged **low-level Linux APIs** including `fork()`, `exec()`, `wait()`, signals, process groups, terminal ownership, and filesystem metadata to implement robust **process and file management** workflows.
- Designed modular components for **UI rendering, file operations, permission handling, and path resolution** while supporting symlinks, FIFOs, sockets, and recursive directory operations.

⇒ **WM Snake Game | C, ncurses, POSIX Threads, Concurrency and Atomics, Linux (2025)**

- Built a real-time **terminal Snake game** from scratch in C with ncurses, implementing **game rendering, collision detection, scoring, pause controls**, and **special timed gameplay events**.
- Architected a **concurrent event-driven game loop** using POSIX threads, mutexes, thread-safe input queues, and C11 **atomic primitives** to **synchronize** rendering, user input, and timer subsystems.
- Engineered binary **game-state persistence** and recovery mechanisms, including serialization of board data structures, timers, score tracking, and runtime state while supporting **dynamic terminal resizing** and fault-tolerant gameplay recovery.

⇒ **wmsh | Rust, Crossterm, Rustix, Shell (2025-26*)**

- Engineered a **Unix-like shell in Rust** featuring custom **terminal input handling, command history, line editing**, built-in commands, and external process execution while directly managing terminal state and user interaction in raw mode.
- Designed a shell grammar and execution architecture supporting **redirections, pipelines, and control-flow operators**, evolving from a flat parser to an AST-based representation to model execution semantics, operator precedence, and future shell extensibility.
- Implemented **low-level I/O redirection and process orchestration** using **Linux file descriptors**, descriptor duplication, and process spawning primitives, emphasizing systems-level resource management, **ownership safety**, and modular interpreter design.

⇒ **WM-Krnl | C, x86 Assembly, OS Development, Memory Management, Filesystems (2026*)**

- Built a **32-bit operating system kernel** from first principles, implementing **bootloading**, interrupt and exception handling, paging, kernel/user privilege separation, **physical memory management**, and software-driven **multitasking** on x86 hardware.
- Architected low-level runtime infrastructure including **process scheduling, context switching**, dynamic kernel memory allocation, virtual memory subsystems, and **protected execution** environments while working directly with CPU control registers, descriptor tables, and hardware interfaces.
- Designed and implemented a **custom filesystem** and storage layer featuring inode-based metadata, extent allocation, bitmap management, directory hierarchies, **executable loading**, and persistent on-disk data structures, gaining hands-on experience with operating system and filesystem design.

⇒ **WM Traffic Simulator | C++, SFML, Kinematics, Graph Theory (2025)**

- Coordinated a **project team of 10 classmates**, managing **task distribution** and component integration to build a real-time **2D traffic simulation** from scratch.
 - Implemented vehicle kinematic behaviors using the **Intelligent Driver Model (IDM)** for acceleration and braking, writing custom bumper-to-bumper distance checks and junction **traffic light state machines** to prevent deadlocks.
 - Developed a **graph-based routing** subsystem using the **A* pathfinding algorithm** with dynamic link weights based on road density, supported by a text-based map parser and **automated CMake** asset deployment pipelines.
-